

← Back to articles

Go Redis WebSocket Real-time

OpenChat's Architecture: Scaling Real-Time Chat with Go, Redis, and WebSockets

Discover how OpenChat scales real-time communication using Go, Redis, and WebSockets. From spam prevention to horizontal scaling, explore the architecture that powers this self-hosted, anonymous chat server.



AZRAF AL MONZIM September 18, 2024 22 views



Table of Contents

OpenChat is a real-time, self-hosted chat server that allows users to create



protests, when internet shutdowns made communication difficult. I envisioned **OpenChat** as a reliable, easy-to-set-up chat server that could operate both on public internet and local networks, ensuring communication channels remain open —even in the face of restrictions.

The Inspiration Behind OpenChat

During the July 2024 protests (also known as the quota movement), some chat servers hosted on BDIX (Bangladesh Internet Exchange) allowed people to communicate without the public internet. However, many of these servers suffered from issues like spamming and frequent downtime, leaving people frustrated. This made me realize the need for a more reliable, scalable, and spam-resistant solution that could handle high traffic and maintain privacy.

Core Objectives

My primary goals while building **OpenChat** were:

- **Easy Setup**: A straightforward, self-hosted solution with minimal configuration.
- **Versatile Usage**: Capable of running on both public and local networks, making it independent of internet availability.
- **Real-Time Communication**: Seamless messaging with minimal latency.
- **Scalability**: Support for a large number of concurrent users across multiple server instances.
- **Robust Spam Prevention**: Implemented measures to combat spamming and abuse.
- **Anonymous Chat Rooms**: Allow users to create and participate in rooms without tracking their identities.
- **Private Rooms**: Secure rooms with password protection for sensitive conversations.

Tech Stack

For simplicity and scalability, I chose the following technologies:

- **Go**: Known for its performance, low memory consumption, and compact



- **Redis**: Used for message storage, pub/sub, and WebSocket scalability. Redis ensures the system can scale horizontally, allowing multiple instances of the server to handle more traffic efficiently.
- **Docker**: For ease of deployment and portability across different environments.
- **Frontend**: I used **Remix.run** for the front-end framework, and **shadcn UI** for styling, ensuring a modern and responsive user experience.

WebSocket Library Experiments

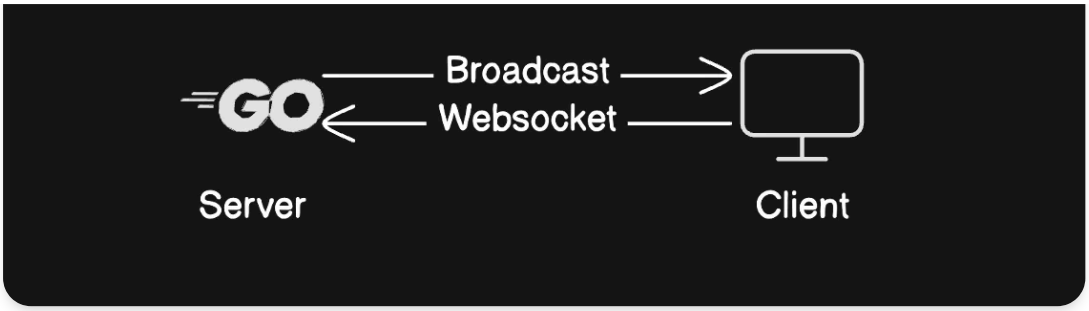
One of the core requirements for **OpenChat** was to establish a reliable WebSocket connection to handle real-time communication. I experimented with multiple Go WebSocket libraries to find the best performance:

1. **Gorilla WebSocket**: Initially used but maxed out at around 1.7k connections per server instance, which wasn't scalable enough for my needs.
2. **Fiber WebSocket**: Managed around 1.8k-2.3k connections per instance, showing some improvement.
3. **nhooyr.io/websocket**: Finally settled on this library as it supported around 3.5k connections per instance, even without optimization. With further tuning, this number could potentially go even higher.

Architecture Evolution

Initial Peer-to-Peer Architecture

At first, I designed **OpenChat** with a simple peer-to-peer architecture where each client connected to a single Go server through a WebSocket. The server would then broadcast messages to all connected clients in real time.



In this setup:

1. Each client connected to a single Go server through a WebSocket.
2. The server would broadcast messages to all connected clients in real-time.
3. All message handling and broadcasting were done in-memory on the server.

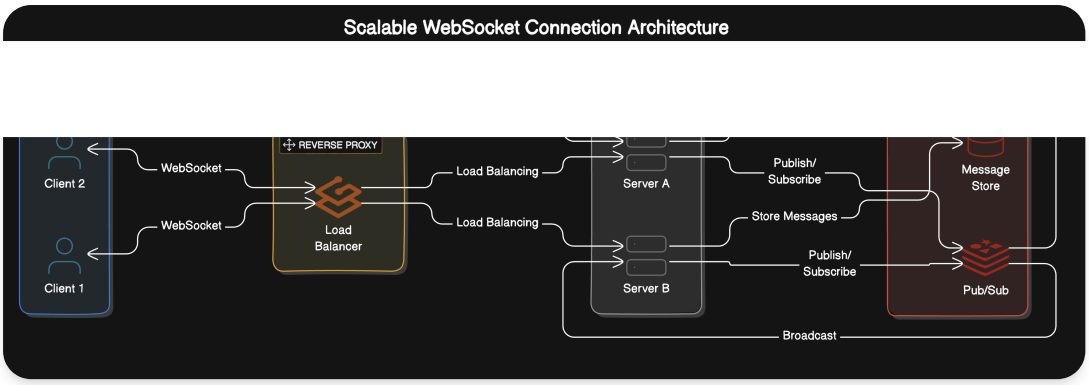
While this worked fine for small-scale scenarios, it became clear that this setup wasn't scalable. The issue was that each server was broadcasting every message to all clients, which quickly became unsustainable as the number of users increased.

Scaling the Architecture

To enable **OpenChat** to handle a large number of users, I needed to scale WebSocket connections horizontally across multiple server instances. The solution was to integrate **Redis** with its pub/sub capabilities to ensure message distribution between server instances.

How It Works

- **Message Flow**: When a client sends a message, the server doesn't broadcast it directly to all connected clients. Instead, the message is forwarded to Redis, which acts as a message broker using pub/sub. Each server instance is subscribed to a specific Redis channel (representing the chat room), ensuring that every server receives and forwards the message to the correct clients.



Key Components of the Scalable Architecture

1. **Horizontal Scaling**: By leveraging Redis for pub/sub, multiple server instances can share the workload, enabling horizontal scaling. This ensures that the chat system can handle thousands of concurrent connections.

2. **Load Balancing**: I used **Nginx** as the load balancer and reverse proxy to distribute WebSocket connections across server instances. For users who prefer it, **Traefik** can also be used as an alternative.

3. **Redis Pub/Sub**: When a client sends a message, instead of broadcasting it directly to all connected clients, the server forwards the message to Redis. Redis then uses its pub/sub mechanism to distribute the message to all subscribed server instances.

4. **Chat Rooms as Channels**: Each chat room is represented as a Redis channel. When a user joins a room, the server instance handling that user's connection subscribes to the corresponding Redis channel.

Message Flow in the Scaled Architecture

1. A client sends a message via WebSocket to its connected server instance.
2. The server instance receives the message and publishes it to the appropriate Redis channel (chat room).
3. Redis broadcasts the message to all server instances subscribed to that channel.
4. Each server instance then forwards the message to its connected clients who are in that chat room.

This architecture allows OpenChat to scale horizontally, distributing the load across multiple server instances while ensuring real-time message delivery to all users in a chat room, regardless of which server instance they're connected to.

Managing WebSocket State



address this, each server instance maintains a list of all its connected clients and their associated chat rooms. This information is stored in the server's memory for quick access.

When a server receives a message from Redis pub/sub:

1. It checks its local list of connected clients.
2. It identifies which of its clients are in the relevant chat room.
3. It forwards the message only to those clients.

This approach ensures efficient message distribution without unnecessary broadcasting.

Storing Messages

In addition to real-time broadcasting, Redis also serves as a temporary storage for chat messages. Server admins can configure how long messages are stored and how many messages to retain. This flexibility allows chat rooms to maintain message history without overwhelming system resources.

Key points about message storage:

- Messages are stored in Redis with a configurable Time-To-Live (TTL).
- Server admins can configure how long messages are stored and how many messages to retain per chat room.
- This flexibility allows chat rooms to maintain message history without overwhelming system resources.
- When a user joins a room, the server can quickly fetch recent messages from Redis and send them to the new participant.

Spam Prevention System

One of the main challenges with public chat servers is preventing spam. **OpenChat** uses multiple techniques to tackle this:

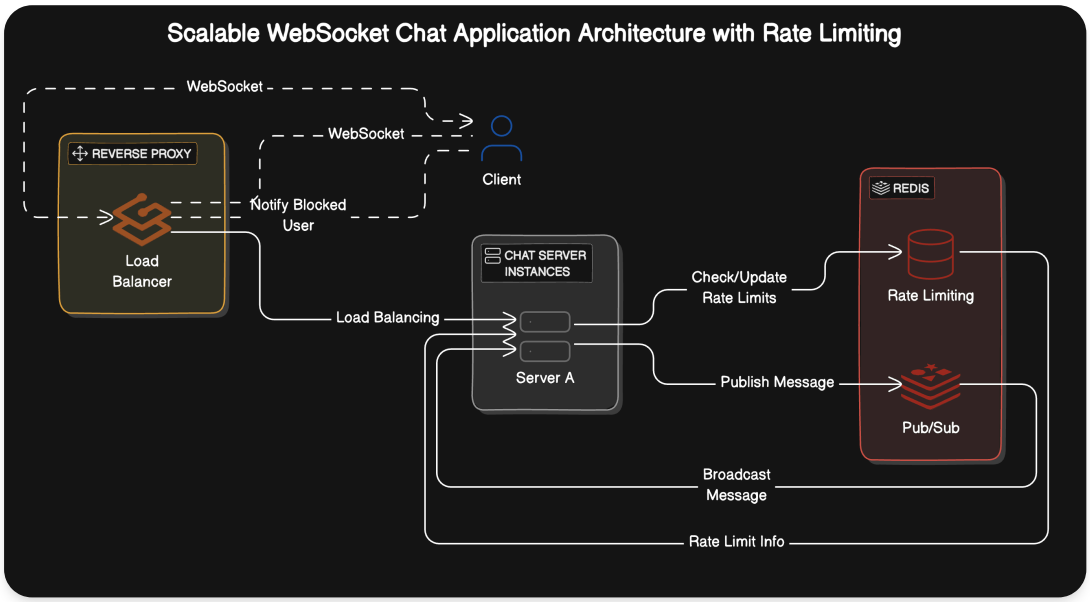
1. **Configuration**: Admins can configure how many messages a user can send within a specific timeframe.

2. **Rate Limiting**: If a user exceeds the configured limit, they are temporarily



3. **Persistent Blocks**: The rate limiting data is stored in Redis, ensuring that even if a server restarts, the block remains in effect.
4. **Real-time Notification**: Blocked users are notified in real-time via their WebSocket connection, informing them of their status and when they can send messages again.

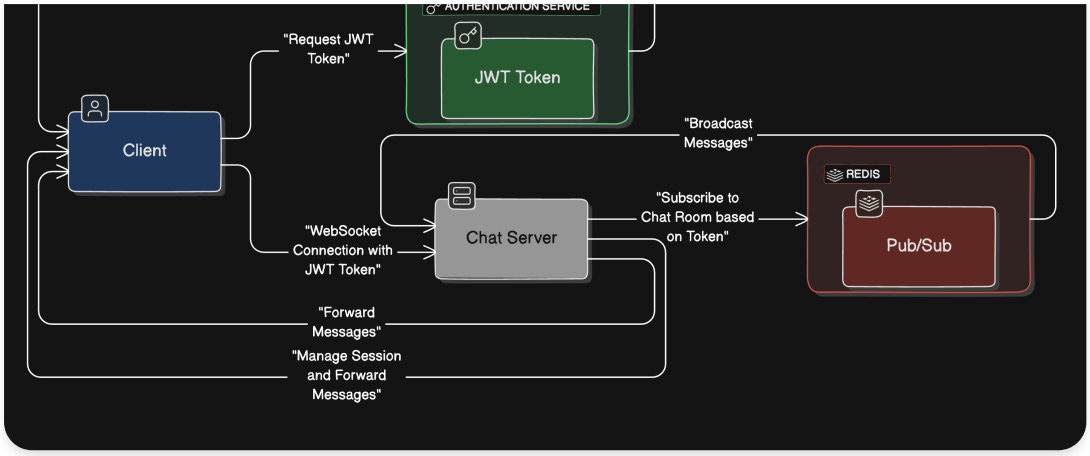
Here's a visual representation of the rate limiting process:



Anonymous, Secure User Authentication

OpenChat ensures user privacy by using **JWT (JSON Web Token)** for session management. When a user connects to the server, a JWT token is generated and stored in their session. The server never stores information about who is in which room, and user identities remain anonymous.

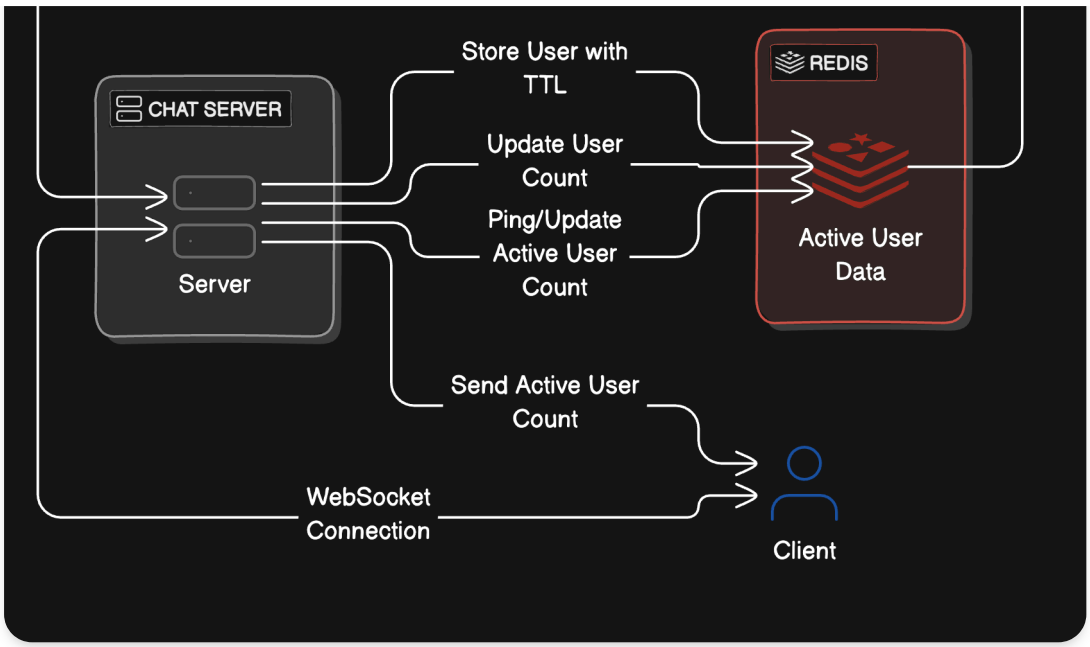
JWT-Based Authentication and User Anonymity Flow in a WebSocket Chat Application



1. When a user connects to the server, a JWT is generated and returned to the user.
2. The JWT contains session information but no personally identifiable information.
3. The token is cryptographically signed, making it tamper-proof.
4. Servers use the JWT to validate the user's session and determine which rooms they have access to.
5. No information about who is in which room is stored server-side, maintaining user anonymity.

Real-Time Active User Count

To display the number of active users in a room, **OpenChat** stores short-lived session data in Redis with a Time-To-Live (TTL). Periodic pings from each user keep this information up to date. The active user count is broadcasted in real time to all connected clients. Like other Redis data, this information persists across server restarts.



1. When a user joins a room, the server stores a short-lived session data in Redis with a Time-To-Live (TTL).
2. Periodic pings from each user's client keep this information up-to-date.
3. The server periodically calculates the active user count for each room based on the Redis data.
4. This count is broadcast in real-time to all connected clients in the room.
5. If a server restarts, the active user information persists in Redis, ensuring continuity.

Message Retention

Messages are not stored permanently in Redis. Instead, the system uses a TTL-based message retention policy. Server admins can configure how long messages are stored and how many messages to retain before they are deleted. This helps maintain a balance between performance and functionality, keeping the system responsive and efficient.

Future Enhancements

The current architecture covers the core functionality of **OpenChat**, but there are several areas I plan to explore in the future:

- **API Documentation**: Detailed documentation for integrating **OpenChat**



- **Continuous Deployment**: Automating deployment processes with Docker and Kubernetes for easier scaling.
- **Advanced Frontend Features**: Enhancing the frontend built with **Remix.run** to handle WebSocket connections more efficiently and improve the user experience.
- **Additional Spam Prevention Mechanisms**: Implementing further measures to improve spam detection and prevention.

Stay tuned as I continue to develop and refine **OpenChat** and release more features in future updates!

Tags: Go Redis WebSocket Real-time Architecture Scalability Docker Anonymous Chat Self-Hosted OpenChat spam prevention JWT

Share:

All Articles

Written by Azraf Al Monzim



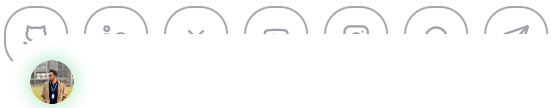
Platform Engineer passionate about building scalable systems and sharing knowledge through writing.

View Profile More Articles

MONZIM

Backend & platform engineer in Dhaka. I help startup teams ship reliable Go and TypeScript backends on Cloudflare and AWS.

Dhaka, BD · Available now



WORK

Services
Projects
ADRs
Labs

WRITING

All articles
RSS feed
Git workshop
GitHub Pages workflow

ME

About
Now
Uses
Talks
Gallery

GET IN TOUCH

Contact form
Book a call
Resume page

hello@monzim.com

+880 1841 500360

Resume (PDF)

Colophon Privacy Terms

© 2026 Azraf Al Monzim · REACT · TANSTACK · TAILWIND · CLOUDFLARE